

역기구학 2 일차 과제 3 레포트

로봇 예비단원 주호진

1) 수치적 역기구학이란?

역기구학 문제를 해결하는 방법은 크게 해석적 방법과 수치적 방법으로 구분한다.

해석적 방법은 기하학적이거나 대수적 방법으로 직접 해를 구하는 것이다.

그러나 해석적으로 해를 유도하기 힘든 복잡한 로봇의 경우 수치적 방법을 사용한다. 수치적 방법은 초기 추정값에서 시작하여 반복적으로 오차를 줄여가며 해를 근사한다. 가장 대표적인 수치해법은 자코비안 행렬을 이용한 방법이다.

자코비안은 각 관절 속도 변화가 EE의 속도 변화에 어떻게 영향을 주는지를 나타내는 미분 행렬로서, 이를 역으로 활용하면 현재 위치에서 목표 위치로 이동하기 위해 관절을 얼마나 변경해야 하는지 계산할 수 있다.

예를 들어 자코비안 역행렬이나 의사역행렬을 통해 관절각 변화를 계산하는 방식이 있다. 이러한 방법은 매 반복 단계마다 목표와 실제 말단 좌표 간 오차를 계산하고, 오차를 줄이는 방향으로 관절각을 수정하면서 수렴해간다. 수치해법은 로봇의 자유도가 높거나 해석적 해법이 어려운 경우 유용하며, 일반적인 6 자유도 이상 로봇 팔, 드론에 부착된 매니퓰레이터 등에서 활용된다.

다만 반복 계산으로 인한 계산 시간 지연 문제가 있으므로, 실시간 제어가 필요한 경우에는 수렴 속도를 높이기 위한 알고리즘 최적화가 필요하다. 현대 로봇 소프트웨어에서는 이러한 수치적 IK를 계산해주는 알고리즘 및 소프트웨어를 구현하여 개발자가 쉽게 활용할 수 있도록 하고 있다.

2) 자코비안 행렬

자코비안 행렬은 다변수 함수의 편미분들로 구성된 행렬로, 로봇에서 관절 속도와 EE 속도 사이의 관계를 나타낸다. 정기구학에서는 관절이 움직이는 속도가 EE 속도로 어떻게 변환되는지를 연결해준다.

역기구학 관점으로 본다면 EE를 목표 위치로 보내려면 관절을 어떻게 움직여야 하는가를 구하면 된다.

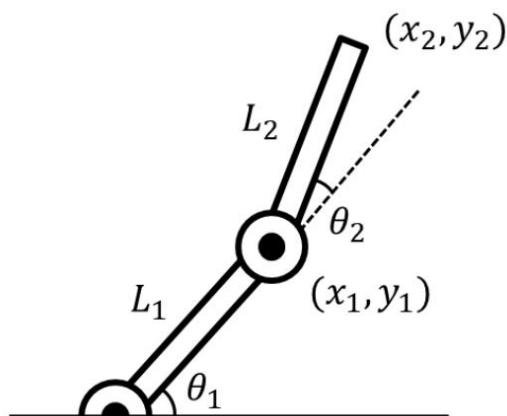
이를 위해 자코비안의 현재 EE와 목표 위치 사이의 거리를 구한 뒤, 자코비안의 역행렬을 이용해 필요한 관절의 각도 변화량을 계산한다. 수치적 역기구학에서는 이 과정의 오차를 계산하여 목표 오차를 줄여가는 방법으로 해를 찾는다.

예를 들어 로봇이 n 개의 자유도를 가지고, EE 공간 좌표 자원이 m 개(ex. 3 차원에서 위치와 자세를 모두 고려하는 경우 $m = 6$, 위치만 고려하는 경우 $m = 3$... 등)일 때 자코비안 행렬은 $m \times n$ 크기의 행렬로 표현된다.

$$J(\theta) = \begin{bmatrix} \frac{\partial x_1}{\partial \theta_1} & \frac{\partial x_1}{\partial \theta_2} & \cdots & \frac{\partial x_1}{\partial \theta_n} \\ \frac{\partial x_2}{\partial \theta_1} & \frac{\partial x_2}{\partial \theta_2} & \cdots & \frac{\partial x_2}{\partial \theta_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_m}{\partial \theta_1} & \frac{\partial x_m}{\partial \theta_2} & \cdots & \frac{\partial x_m}{\partial \theta_n} \end{bmatrix}$$

θ 는 각 관절의 각도를 의미하고, x 는 EE 의 위치(x, y, z) 및 자세(ϕ, θ, ψ)를 의미한다.

아래 간단한 2 차원 2 자유도 매니플레이터 예제를 통해 자코비안을 알아보겠다.



해당 매니플레이터의 x_2, y_2 값은 다음과 같이 표현된다.

$$\begin{aligned} x_2 &= L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ y_2 &= L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

x_2, y_2 를 편의상 x, y 로 표기한다면 매니플레이터의 말단 속도 $\dot{x}$ 헛, $\dot{y}$ 헛은 각 관절의 속도 $\dot{\theta}_1$ 헛, $\dot{\theta}_2$ 헛에 의해 다음과 같이 표현한다.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \mathbf{J}(\theta_1, \theta_2) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

여기서 $\mathbf{J}$ 는 각 위치를 관절 각도에 대해 편미분한 행렬이므로, 위 예제에 적용하면 다음과 같이 나오게 된다.

$$\mathbf{J} = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

여기까지 세운 식이 관절이 움직이는 속도가 EE 속도로 어떻게 변환되는지에 대한 정기구학 관점이고, 이걸 반대로 역기구학의 관점으로 보게 된다면 다음과 같이 표현할 수 있다.

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \mathbf{J}^{-1}(\theta_1, \theta_2) \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}$$

여기서 반복계산을 하며 오차를 줄여나가는 수치적 관점을 적용하게 된다면 다음과 같이 표현된다.

$$\begin{bmatrix} \Delta \theta_1 \\ \Delta \theta_2 \end{bmatrix} = \mathbf{J}^{-1}(\theta_1, \theta_2) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

자코비안의 역행렬은 첫날에 공부했던 역행렬 식에 의해 다음과 같이 쓸 수 있다.

$$\mathbf{J}^{-1} = \frac{1}{L_1 L_2 \sin \theta_2} \begin{bmatrix} L_2 \cos(\theta_1 + \theta_2) & L_2 \sin(\theta_1 + \theta_2) \\ -L_1 \cos \theta_1 - L_2 \cos(\theta_1 + \theta_2) & -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}$$

역행렬에 곱해진 자코비안의 행렬식을 보면 L_1, L_2 은 상수이므로 큰 문제가 없지만 $\sin \theta_2$ 는 θ 값에 따라서 역행렬이 존재하지 않을 수도 있다. 이러한 지점을 특이점이라고 하는데 아래 항목에서 이어서 서술하겠다.

3) Singularity

Singularity(특이점)은 로봇이 특정 자세를 취할 때 기하학적으로 자유도가 상실되거나 특정 방향으로 이동이 불가능해지는 상태를 말한다. 예를 들면 다리를 짝 뻗을 때 종아리를 무릎 앞방향으로 접을 수 없는 것과 마찬가지로의 상황이다.

위 2 의 자코비안 역행렬 예제식에서 봤듯이 각도에 따라 어떤 특이점 근처에 도달하면 자코비안의 역행렬의 분모가 0 에 가까워지게 되고, 경우에 따라서는 분모가 0 이 되는 값이 발생한다. 분모가 0 에 가까워지면 값이 무한대로 튀기 때문에 $\Delta\theta$ 가 무한대로 발산하는 경우가 생긴다. $\Delta\theta$ 가 무한대로 발산하는 경우 의도치 않은 동작이 일어나거나, 하드웨어에 손상이 생길 수 있다.

또 다른 특이점은 일반적으로 두개 혹은 그 이상의 조인트(관절) 축 방향이 일치할 때 생긴다. 곧 관절의 방향 벡터가 겹치게 된다는 말인데, 독립적인 관절끼리 방향 벡터가 겹치는 경우 특정 방향으로 움직일 수 있는 자유도를 순간적으로 상실하게 된다. 독립된 관절이 마치 순간적으로 하나의 관절이 되어버리는 셈이다.

역행렬이 존재하기 위해서는 원래의 행렬과 일대일 대응이 되어야 하는데 위와 같은 상황이 될 경우 독립이 보장되지 않기 때문에, 일대일 대응이 깨지면서 역행렬을 구할 수 없게 되는 것이다.

행렬에서 역행렬을 구할 수 없다는 것은 곧 행렬식이 0 이라는 뜻이기 때문에 앞선 케이스와 같이 $\Delta\theta$ 가 무한대로 발산해버린다.

Singularity 방지 방법

위와 같은 Singularity가 일어나지 않으려면 예외처리와 같은 방지가 필요하다. 예외처리는 아래에 대표적인 3 가지를 기재하겠다.

- DLS(Damped Least Squares)

분모가 0 에 가까워져 속도가 무한대로 발산하지 않게 λ^2 라는 아주 작은 값을 가진 댐핑 계수를 더해 안정성을 주는 기법이다. 이렇게 하면 기존에 아예 분모가 0 인 상태에서도 λ 가 아주 작은 값을 가지고 있기 때문에 속도가 무한대로 발산하지 않게 방지해준다.

발산을 막아주는 장점이 있지만, 특이점 부근을 통과할 때 로봇 끝단이 원래 가려던 경로에서 벗어나게 되는 오차가 발생한다.

- Null space 활용

로봇의 관절 개수가 작업 공간의 자유도(차원)보다 많을 때 그 여유분을 여유 자유도라고 한다.

로봇이 여유 자유도가 생기면 Null space 라는 것이 생긴다. 로봇공학에서의 Null space 는 EE 을 고정하고 자유롭게 움직일 수 있는 공간을 의미한다.

만약 여유 자유도가 있고 Null space 가 있다면, EE 를 고정한 상태(원하는 작업을 방해받지 않고)로 특이점 지점을 피해 지나가도록 설계할 수 있다.

- 최대 허용 값 설정

수치적 IK 결과로 나온 값이 너무 크다면 최대 허용 값을 미리 설정해서 속도가 해당 값 이상으로 넘어가게 됐을 때 최대 허용 값을 적용하는 방법이다.

다만 수치를 강제로 조정하기 때문에 관절의 비율이 틀어져 원래의 궤도에서 틀어질 수 있는 단점이 있다.